CALIFORNIA STATE UNIVERSITY
LONG BEACH

CECS 341: Computer Architecture and Organization

Hossein Sayadi, PhD

Department of Computer Engineering & Computer Science
California State University, Long Beach

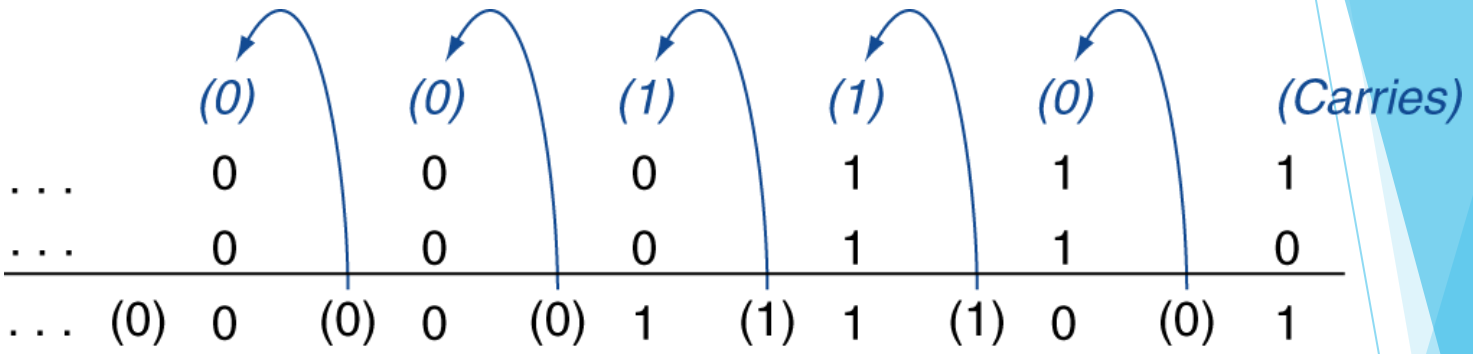
Fall 2025

Lecture Outline

- Intro to Computer Arithmetic
  - Operations on integers
  - Overflow
  - Floating-point real numbers
- Introduction to Register Transfer Language (RTL)
  - Describing CPU micro-operations using RTL
  - Describing a processor using RTL
- Describing the MIPS using RTL

Integer Addition

- Example: 7 + 6



- Overflow if result out of range
  - Adding +ve and –ve operands, no overflow
  - Adding two +ve operands
    - Overflow if result sign is 1
  - Adding two –ve operands
    - Overflow if result sign is 0

Integer Subtraction

- Add negation of second operand
- Example: 7 – 6 = 7 + (–6)

```
+7:     0000 0000 … 0000 0111
–6:     1111 1111 … 1111 1010
+1:     0000 0000 … 0000 0001
```

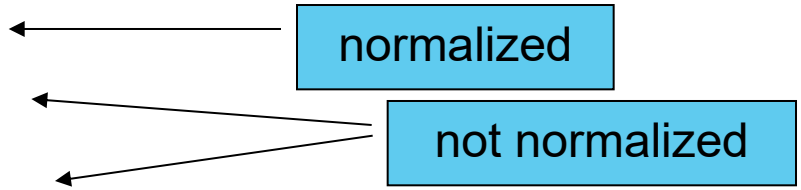
- Overflow if result out of range
  - Subtracting two +ve or two –ve operands, no overflow
  - Subtracting +ve from –ve operand
    - Overflow if result sign is 0
  - Subtracting –ve from +ve operand
    - Overflow if result sign is 1

Dealing with Overflow

- Some languages (e.g., C) ignore overflow
  - Use MIPS `addu`, `addui`, `subu` instructions
- Other languages (e.g., Ada, Fortran) require raising an exception
  - Use MIPS `add`, `addi`, `sub` instructions
  - On overflow, invoke exception handler
    - Save PC in exception program counter (EPC) register
    - Jump to predefined handler address
    - `mfc0` (move from coprocessor reg) instruction can retrieve EPC value, to return after corrective action

Floating Point

- Representation for non-integral numbers
  - Including very small and very large numbers
- Like scientific notation
  - -2.34×10^{56}
  - $+0.002 \times 10^{-4}$
  - $+987.02 \times 10^{9}$
- In binary
  - $\pm 1.xxxxxxx_2 \times 2^{yyyy}$
- Types `float` and `double` in C
- Defined by IEEE Std 754-1985
- Two representations
  - Single precision (32-bit)
  - Double precision (64-bit)



IEEE Floating-Point Format

single: 8 bits
double: 11 bits

single: 23 bits
double: 52 bits



$$x = (-1)^S \times (1 + \text{Fraction}) \times 2^{(\text{Exponent} - \text{Bias})}$$

- S: sign bit (0 ⇒ non-negative, 1 ⇒ negative)
- Normalize significand: 1.0 ≤ |significand| < 2.0
  - Always has a leading pre-binary-point 1 bit, so no need to represent it explicitly (hidden bit)
  - Significand is Fraction with the "1." restored
- Exponent: excess representation: actual exponent + Bias
  - Ensures exponent is unsigned
  - Single: Bias = 127; Double: Bias = 1023

Floating-Point Example

- Represent –0.75
  - $-0.75 = (-1)^1 \times 1.1_2 \times 2^{-1}$
  - S = 1
  - Fraction = $1000\ldots00_2$
  - Exponent = –1 + Bias
    - Single: –1 + 127 = 126 = 01111110_2
    - Double: –1 + 1023 = 1022 = 01111111110_2
- Single: 10111111001000…00
- Double: 10111111111101000…00

Floating-Point Example

- What number is represented by the single-precision float
  1100000001000…00
  - S = 1
  - Fraction = $01000\ldots00_2$
  - Exponent = $10000001_2 = 129$
- $x = (-1)^1 \times (1 + 01_2) \times 2^{(129 - 127)}$
  $= (-1) \times 1.25 \times 2^2$
  $= -5.0$

Floating-Point Addition

- Consider a 4-digit decimal example
  - $9.999 \times 10^{1} + 1.610 \times 10^{-1}$
- 1. Align decimal points
  - Shift number with smaller exponent
  - $9.999 \times 10^{1} + 0.016 \times 10^{1}$
- 2. Add significands
  - $9.999 \times 10^{1} + 0.016 \times 10^{1} = 10.015 \times 10^{1}$
- 3. Normalize result & check for over/underflow
  - 1.0015×10^{2}
- 4. Round and renormalize if necessary
  - 1.002×10^{2}

Introduction to RTL

Register Transfer Language (RTL)

Register Transfer Language (RTL)

- Register transfer language
  - A symbolic language
  - A convenient tool for ***describing*** the internal organization of a computer.
  - Can also be used to facilitate the ***design*** process of digital systems.
- RTL can be used to describe the sequence of *micro-operations*, or actions, needed to implement any processor instruction.
- RTL is a concise and precise means of describing the micro-operations that can be performed on the data stored in the registers of a processor.
- RTL is a simple, human-oriented language to specify the operations, register communication, and timing of the steps implemented by the CPU for each user programmable instruction.

Registers and RTL Expressions

- Registers are denoted by *upper-case* letters.
  - Optionally followed by numbers or letters.
- RTL expressions
  - Identify the registers to be manipulated
  - Specify the micro-operations to be performed on the registers.
- Basic components of RTL expressions:

Symbol	Description	Example
Letters and Numbers	Denotes a register	R1, PC, MAR
Parentheses	Denotes a part of a register	R1(5 – 0), R2(17), MDR(31 – 16)
Left arrow	Denotes transfer of information	R1 <- R2
Comma	Separates two micro-operations	R1 <- R2, R3 <- R4
Colon	Separates control from expression	P: R1 <- R2

RTL Micro-operations

- Each *micro-operation*, or action, is completed in one clock cycle.

- Concurrent (parallel) micro-operations
  - Some micro-operations can be implemented at the same time.
  - They are implemented in different parts of the processor.
  - They are independent of one another.
  - They are completed in the same clock cycle.
- Sequential micro-operations
  - Other micro-operations must be implemented in sequence.
  - They are dependent on one another.
  - Each requires a clock cycle.

Common RTL Micro-operations

- Data Transfer

RTL	Description
R1 <- R2	Transfer (copy) contents of register R2 to register R1 (Requires one CPU bus)
R1 <- R2, R3 <- R4	Concurrent transfer Transfer (copy) contents of R2 to R1 and contents of R4 to R3. (Requires two CPU buses)
P: R1 <- R2	Conditional transfer Transfer contents of R2 to R1, if (P = 1).
T: R1 <- R2, R3 <- R4	Conditional, concurrent transfer Transfer contents of R2 to R1 and contents of R4 to R3, if (T = 1).

Data transfer micro-operations are implemented using the internal CPU bus(es).

Common RTL Micro-operations

- Arithmetic

RTL	Description
R3 <- R1 + R2	Contents of R1 plus contents of R2 transferred to R3
R3 <- R1 – R2	Contents of R1 minus contents of R2 transferred to R3
R2 <- R2’	1’s Complement of R2
R2 <- R2’ + 1	2’s Complement of R2 (negate)
R3 <- R1 + R2’ + 1	R1 plus 2’s Complement of R2 (R1 – R2) transferred to R3
R1 <- R1 + 1	Increment contents of R1 by 1
R1 <- R1 – 1	Decrement contents of R1 by 1

Arithmetic micro-operations are implemented by the ALU.

Multiplication and division are not considered micro-operations.

Common RTL Micro-operations

- Logic

RTL	Description
R3 <- R1 and R2	Contents of R1 *ANDed with* contents of R2 transferred to R3
R3 <- R1 or R2	Contents of R1 *ORed with* contents of R2 transferred to R3
R1 <- R1'	Complement of R1 (NOT)
R3 <- R1 xor R2	Contents of R1 *XORed with* contents of R2 transferred to R3
R3 <- (R1 and R2)'	Contents of R1 *NANDed with* contents of R2 transferred to R3
R3 <- (R1 or R2)'	Contents of R1 *NORed with* contents of R2 transferred to R3
R3 <- (R1 xor R2)'	Contents of R1 *XNORed with* contents of R2 transferred to R3

Logic micro-operations are implemented by the ALU.

Most systems only implement AND, OR, NOT, and XOR.

Memory Transfer

- Read

A read operation transfers data ***from memory to a register***.

RTL	Description
MAR <- *address*	Copy address into Memory Address Register (MAR)
MDR <- M[MAR]	Transfer data from memory at address in MAR to Memory Data Register (MDR)

- Write

A write operation transfers data ***from a register to memory***.

RTL	Description
MAR <- *address*	Copy address into Memory Address Register (MAR)
M[MAR] <- MDR	Transfer data from Memory Data Register (MDR) to memory at address in MAR.

Describing the MIPS Architecture using RTL

MIPS Instruction Set Architecture

- MIPS is a 32-bit processor
  - ALU operands are 32 bits
  - Registers are 32 bits
- Registers
  - Thirty-two, 32-bit registers (general purpose, SP, FP, RA, etc.)
  - Program Counter (PC), Instruction Register (IR)
  - Memory Address Register (MAR), Memory Data Register (MDR)
- Instruction Formats: R-type, I-type, and J-type
- Memory system
  - 32-bit address
  - Byte-addressable memory

Registers and Memory

Registers	RTL Description
Thirty-two 32-bit registers (Register File)	R[0..31](31 – 0)
Program Counter (PC)	PC(31 – 0)
Instruction Register (IR)	IR(31 – 0)
Memory Address Register (MAR)	MAR(31 – 0)
Memory Data Register (MDR)	MDR(31 – 0)

32 registers

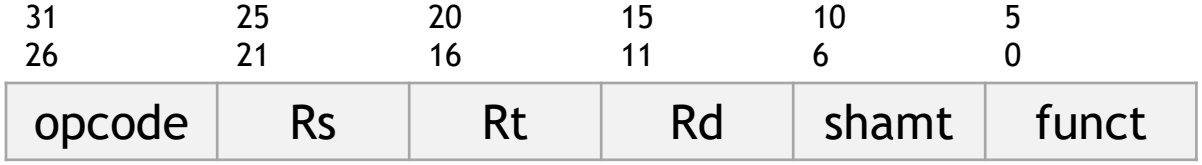
32 bits (MSb = bit 31)

Memory	RTL Description
Address width = 32 bits Byte-addressable	M[0..2^{32} – 1](7 – 0)

2^{32} memory locations

1 byte (8 bits)

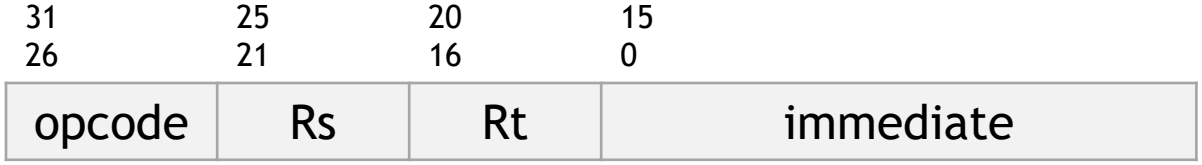
Instruction Formats: R-type



R-type

Field name	Bits of Instruction Register (IR)
opcode(5 – 0)	IR(31 – 26)
Rs(4 – 0)	IR(25 – 21)
Rt(4 – 0)	IR(20 – 16)
Rd(4 – 0)	IR(15 – 11)
shamt(4 – 0)	IR(10 – 6)
funct(5 – 0)	IR(5 – 0)

Instruction Formats: I-type



I-type

Field name	Bits of Instruction Register (IR)
opcode(5 – 0)	IR(31 – 26)
Rs(4 – 0)	IR(25 – 21)
Rt(4 – 0)	IR(20 – 16)
Immediate (15 – 0)	IR(15 – 0)

Instruction Formats: J-type

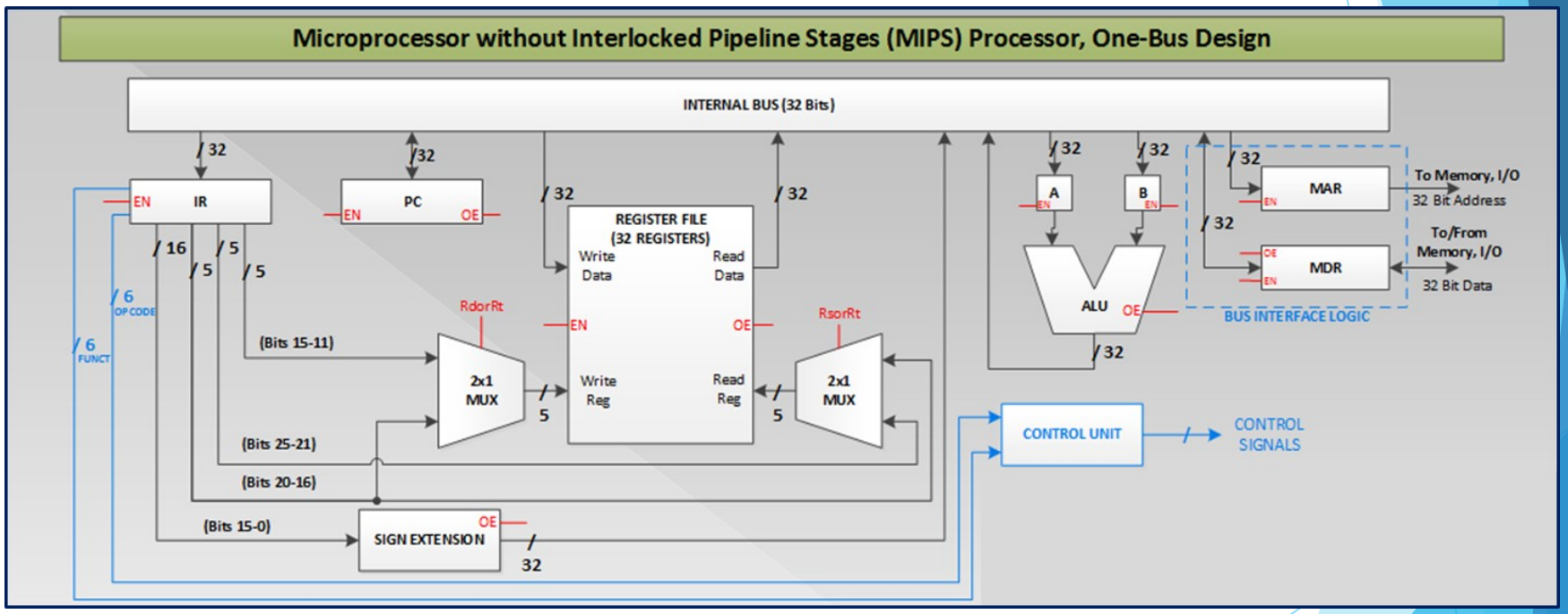


J-type

Field name	Bits of Instruction Register (IR)
opcode(5 – 0)	IR(31 – 26)
address(25 – 0)	IR(25 – 0)

RTL Development

MIPS: One-bus Organization



MIPS (One-bus): Add

- Using RTL, specify the micro-operations needed to execute the `add` instruction on the MIPS One-bus organization.
  - Assume that the instruction has been fetched and is in the IR.
- Identify the control signals that must be *asserted* for each micro-operation.
  - Specify the value where necessary.

Clock Cycle	RTL	Description	Control Signals
1	A <- RF[Rs]	Transfer contents of Rs register in RF to Reg A.	A_{EN}, RF_{OE}, RSorRT = 0
2	B <- RF[Rt]	Transfer contents of Rt register in RF to Reg B.	B_{EN}, RF_{OE}, RSorRT = 1
3	RF[Rd] <- A + B	Add A and B and store result in Rd register in RF.	RF_{EN}, RDorRT = 0, ALU_{OE}, ALU_{CNTL} = 0010

MIPS (One-bus): Sub

- Using RTL, specify the micro-operations needed to execute the `sub` instruction on the MIPS One-bus organization.
  - Assume that the instruction has been fetched and is in the IR.
- Identify the control signals that must be *asserted* for each micro-operation.
  - Specify the value where necessary.

Clock Cycle	RTL	Description	Control Signals
1	A <- RF[Rs]	Transfer contents of Rs register in RF to Reg A.	A_{EN}, RF_{OE}, RSorRT = 0
2	B <- RF[Rt]	Transfer contents of Rt register in RF to Reg B.	B_{EN}, RF_{OE}, RSorRT = 1
3	RF[Rd] <- A - B	Subtract A and B and store result in Rd register in RF.	RF_{EN}, RDorRT = 0, ALU_{OE}, ALU_{CNTL} = 0110